Integral Numeric Types

C# Type	.NET Type	Size (Bytes)	Sign	Range / Description
`sbyte`	`SByte`	1	Signed	`-128` to `127`
`byte`	`Byte`	1	Unsigned	`0` to `255`
`short`	`Int16`	2	Signed	`-32,768` to `32,767`
`ushort`	`UInt16`	2	Unsigned	`0` to `65,535`
`int`	`Int32`	4	Signed	`-2,147,483,648` to `2,147,483,647` (Default for integers)
`uint`	`UInt32`	4	Unsigned	`0` to `4,294,967,295`
`long`	`Int64`	8	Signed	`-9,223,372,036,854,775,808` to `9,223,372,036,854,775,807`
`ulong`	`UInt64`	8	Unsigned	`0` to `18,446,744,073,709,551,615`
`nint`	`IntPtr`	Platform	Signed	Native integer (32-bit on 32-bit apps, 64-bit on 64-bit apps)
`nuint`	`UIntPtr`	Platform	Unsigned	Native unsigned integer

Floating-Point & Decimal Types

C# Type	.NET Type	Size (Bytes)	Precision	Range	Best Used For
`float`	`Single`	4	6-9 digits	`±1.5 × 10^-45` to `±3.4 × 10^38`	Graphics, memory-sensitive arrays
`double`	`Double`	8	15-17 digits	`±5.0 × 10^-324` to `±1.7 × 10^308`	General scientific/math (Default for decimals)
`decimal`	`Decimal`	16	28-29 digits	`±1.0 × 10^-28` to `±7.9 × 10^28`	Financial and currency calculations

Numeric Literals & Suffixes

By default, integer literals are treated as `int`, and floating-point literals as `double`. Use suffixes to enforce specific types:

- `L` or `l`: `long` (e.g., `long x = 42L;`)
- `UL` or `ul` / `LU`: `ulong` (e.g., `ulong x = 42UL;`)
- `F` or `f`: `float` (e.g., `float f = 3.14f;`)
- `M` or `m`: `decimal` (e.g., `decimal money = 19.99m;`)
- **Underscores (`_`)**: Digit separators for readability (e.g., `int millions = 1_000_000;`)
- **Binary (`0b`) & Hex (`0x`)**: `C# int binary = 0b1010; // 10 int hex = 0x2A; // 42`

Default Values & Conversions

- **Default Value**: For all numeric types, `default(T)` or `new T()` evaluates to `0` (or `0.0`).
- **Implicit Conversions**: Smaller types automatically convert to larger types without data loss (`int` → `long` → `double`).
- **Explicit Conversions (Casting)**: Required when narrowing or risking overflow. Can use explicit casts or `checked` blocks to throw exceptions on overflow: `C# long bigNum = 500L; int smallNum = (int)bigNum; // Explicit cast`